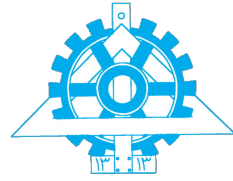




University of Tehran
College of Engineering
School of Electrical and
Computer Engineering



Machine Learning

Assignment 3

Full Name

Sepanta Ghonoodi

Student ID

810102483

December 10, 2025

Abstract

This report details the implementation and analysis of core machine learning algorithms for classification tasks. First, a complete Support Vector Machine (SVM) pipeline was developed for the Pima Indians Diabetes dataset, examining the effects of feature scaling, kernel selection, and class balancing on model performance. The analysis highlighted the superior generalization of the RBF kernel and the necessity of handling class imbalance to improve recall. Second, the primal formulation of the soft-margin SVM was implemented as a Quadratic Programming (QP) problem, directly solving for the optimal hyperplane and validating the solution on the Social Network Ads dataset. Finally, a Decision Tree classifier based on the ID3 algorithm was constructed from scratch to solve the "Play Tennis" problem. This implementation addressed the high cardinality trap, extended the algorithm to handle continuous features, and demonstrated the impact of noise and pruning on model overfitting.

Contents

Abstract	i
1 Question 1: Answer These Questions	1
1.1 Preference for Dual Formulation with Kernels	1
1.2 Support Vectors and Dual Coefficients	1
1.3 Mathematical Explanation of the Kernel Trick	2
1.4 Effect of Kernel Choice	2
2 Question 2: RBF kernel feature space intuition	3
2.1 Mathematical Derivation	3
2.2 Taylor Series Expansion	4
2.3 Interpretation as an Inner Product	4
3 Question 3: SVM dual symmetric 4-point example	4
3.1 Dual Formulation Setup	5
3.2 Computing the Gram Matrix	5
3.3 Solving for Optimal α	6
3.4 Computing Primal Parameters w and b	6
4 Question 4: Harder Kernel-Trick Question (Explicit cubic feature map)	7
4.1 Expansion and Dimensionality	7
4.2 Explicit Feature Map Construction	8
4.3 Verification of Equality	8
5 Question 5: The Mechanics of Splitting (ID3)	9
5.1 Root Entropy Calculation	9
5.2 Information Gain Calculation	10
5.3 Tree Construction	11
5.4 Comparison: Top-down vs. Bottom-up	11
6 Question 6: Optimization Geometry & Jensen's Inequality	11
6.1 Proof of Concavity	12
6.2 Application of Jensen's Inequality	13
6.3 Optimization Trap (Hill Climbing)	13
7 Question 7: The Failure of Greedy Search (XOR)	15
7.1 The Greedy Trap Calculation	15
7.2 Existence of a Perfect Tree	16
7.3 Inductive Bias	16
8 Question 8: From Data to Diagnosis: Diabetes Classification Using SVM	17
8.1 Data Preprocessing and Exploration	18
8.2 Linear SVM Classification	19
8.3 Comparison of Different Kernels	20
8.4 Hyperparameter Tuning	20
8.5 Dimensionality Reduction and Visualization	22
8.6 Class Imbalance Analysis	23

9	Question 9: The QP-SVM Primal Challenge: Optimizing and Visualizing the Soft-Margin Hyperplane SVM	25
9.1	Data Preprocessing	25
9.2	QP Formulation and Solution	25
9.3	Model Evaluation	26
9.4	Final Deliverable: Visualizing the Decision Boundary	26
10	Question 10: Project: Decision Tree Classifier	26
10.1	Data Loading & The High Cardinality Trap	26
10.2	Entropy & Information Gain Engine	28
10.3	Handling Continuous Features	28
10.4	Recursive Tree Construction & Evaluation	29
10.5	Overfitting & Pruning Analysis	29
	References	29

List of Figures

1	Zero value counts before cleaning and final dataset shape	18
2	Confusion Matrix and Accuracy for Linear SVM (Scaled vs Unscaled)	19
3	Confusion Matrices for Max Accuracy Scenario	20
4	Confusion Matrices for Max F1 Scenario	21
5	Final Performance Summary Table	21
6	Grid Search Results and Best Parameters	22
7	2D PCA Projection of Training Data	22
8	SVM Decision Boundary on PCA Reduced Data	23
9	Accuracy on PCA Reduced Data	23
10	Class Distribution and Classification Reports (Standard vs Balanced)	24
11	Data Loading and Shape Verification	25
12	Optimal Parameters obtained from QP Solver	25
13	Custom QP-SVM Model Accuracy	26
14	QP-SVM Decision Boundary and Margins	27
15	Dataset Head and Structure	27
16	Information Gain for Day_ID	27
17	Root Node Feature Selection	28
18	Optimal Split for Temperature	28
19	Model Performance Metrics	29
20	Tree Depth and Leaf Count Analysis	29
21	Tree Size vs. Accuracy (Pruning Simulation)	30

Question 1: Answer These Questions

- (a) Why is the dual formulation of SVM usually preferred when kernels are used? Provide two reasons.
- (b) Explain the statement: “Only support vectors influence the final SVM classifier.” Why do non-support vectors have zero dual coefficients?
- (c) Explain mathematically why the kernel trick allows SVMs to operate in infinite-dimensional spaces without computing the feature map explicitly.
- (d) Explain how the choice of kernel (linear, polynomial, RBF) affects the decision boundary, model capacity, and tendency to overfit.

1.1 Preference for Dual Formulation with Kernels

The dual formulation is preferred for two primary reasons:

- (a) **Independence from Feature Dimensionality:** In the primal formulation, the optimization involves finding a weight vector w whose dimensionality matches the feature space (d). When using kernels (especially RBF), the feature space can be infinite-dimensional ($d \rightarrow \infty$). The dual formulation optimization depends only on the number of training samples (N), specifically finding N Lagrange multipliers α_i , making it computationally feasible regardless of the feature space dimensionality.
- (b) **Explicit Dot Products (The Kernel Trick):** The objective function in the dual formulation depends solely on the dot products between sample pairs $\langle x_i, x_j \rangle$. This structure allows us to replace the dot product in the high-dimensional space $\langle \phi(x_i), \phi(x_j) \rangle$ directly with a kernel function $K(x_i, x_j)$ without ever needing to compute the coordinates of the data in that high-dimensional space.

1.2 Support Vectors and Dual Coefficients

The statement “Only support vectors influence the final SVM classifier” is a direct consequence of the **KKT (Karush-Kuhn-Tucker) Complementary Slackness condition**.

Mathematically, for the constraints $y_i(w^T x_i + b) \geq 1$, the complementary slackness condition states:

$$\alpha_i [y_i(w^T x_i + b) - 1] = 0$$

This equation implies two possibilities for every data point i :

- **Non-Support Vectors:** The point is correctly classified and lies beyond the margin (safe region), so $y_i(w^T x_i + b) - 1 > 0$. For the product to equal zero, the Lagrange multiplier α_i **must** be zero. Consequently, these points do not contribute to the weight vector summation $w = \sum \alpha_i y_i x_i$.
- **Support Vectors:** The point lies exactly on the margin, so $y_i(w^T x_i + b) - 1 = 0$. In this case, α_i can be non-zero ($\alpha_i > 0$).

1.3 Mathematical Explanation of the Kernel Trick

The SVM dual objective function seeks to maximize:

$$L(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \langle \phi(x_i), \phi(x_j) \rangle$$

Here, $\phi(x)$ is a mapping from the input space to a higher (potentially infinite) dimensional feature space $\mathcal{H}$. If we were to solve this explicitly, we would need to compute the vector $\phi(x)$, which is impossible if the dimension is infinite.

However, Mercer's Theorem guarantees that for a valid positive semi-definite kernel function K , there exists a mapping ϕ such that:

$$K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$$

By substituting $K(x_i, x_j)$ into the objective function:

$$L(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

We can solve the optimization and make predictions using the decision function $f(x) = \text{sign}(\sum \alpha_i y_i K(x_i, x) + b)$ using only the scalar values returned by the kernel function K . We never explicitly calculate or store the infinite-dimensional vector $\phi(x)$.

1.4 Effect of Kernel Choice

- **Linear Kernel:**
 - **Decision Boundary:** Strictly linear (a flat hyperplane).
 - **Model Capacity:** Low. Restricted to linear separations.
 - **Overfitting:** Low tendency to overfit (high bias), but likely to underfit if data is complex.
- **Polynomial Kernel:**

- **Decision Boundary:** Curved boundaries; complexity depends on degree d .
 - **Model Capacity:** Moderate to High. Capacity increases as d increases.
 - **Overfitting:** Moderate risk. Higher degrees create “wiggly” boundaries that can fit noise.
- **RBF (Gaussian) Kernel:**
 - **Decision Boundary:** Highly flexible. Can form closed loops or localized “islands” around specific classes.
 - **Model Capacity:** Very High (Infinite VC dimension).
 - **Overfitting:** High tendency to overfit. If bandwidth σ is small (or γ is large), the model essentially memorizes data points.

Question 2: RBF kernel feature space intuition

Explain why the Gaussian (RBF) kernel $K(x, z) = \exp(-||x - z||^2 / (2\sigma^2))$ can be interpreted as an inner product in an infinite-dimensional Hilbert space.

2.1 Mathematical Derivation

To demonstrate that the RBF kernel corresponds to an inner product in an infinite-dimensional space, we can expand the kernel function using the Taylor series.

First, we expand the squared Euclidean distance term in the exponent:

$$||x - z||^2 = \langle x - z, x - z \rangle = ||x||^2 + ||z||^2 - 2\langle x, z \rangle$$

Substituting this back into the kernel definition (assuming $\sigma = 1$ for simplicity, or letting $\gamma = 1/2\sigma^2$):

$$K(x, z) = \exp(-\gamma||x - z||^2) = \exp(-\gamma(||x||^2 + ||z||^2 - 2\langle x, z \rangle))$$

This expression can be factored into three parts:

$$K(x, z) = \exp(-\gamma||x||^2) \cdot \exp(-\gamma||z||^2) \cdot \exp(2\gamma\langle x, z \rangle)$$

The first two terms, $C_x = \exp(-\gamma||x||^2)$ and $C_z = \exp(-\gamma||z||^2)$, act as scaling constants for their respective vectors. The crucial term is the interaction term $\exp(2\gamma\langle x, z \rangle)$.

2.2 Taylor Series Expansion

We apply the Taylor series expansion for the exponential function, $e^u = \sum_{k=0}^{\infty} \frac{u^k}{k!}$, to the interaction term:

$$\exp(2\gamma\langle x, z \rangle) = \sum_{k=0}^{\infty} \frac{(2\gamma\langle x, z \rangle)^k}{k!} = \sum_{k=0}^{\infty} \frac{(2\gamma)^k}{k!} (\langle x, z \rangle)^k$$

Using the multinomial expansion, the term $(\langle x, z \rangle)^k = (\sum_{j=1}^d x_j z_j)^k$ expands into a sum of all possible monomials of degree k .

2.3 Interpretation as an Inner Product

Because the expansion involves a sum from $k = 0$ to ∞ , the kernel $K(x, z)$ can be written as an inner product of two vectors $\phi(x)$ and $\phi(z)$ that contain an infinite number of components (monomials of all degrees):

$$K(x, z) = \langle \phi(x), \phi(z) \rangle_{\mathcal{H}}$$

where the feature map $\phi(x)$ has the form:

$$\phi(x) = \exp(-\gamma\|x\|^2) \left[1, \sqrt{\frac{2\gamma}{1!}}x, \sqrt{\frac{(2\gamma)^2}{2!}}x^2, \dots, \sqrt{\frac{(2\gamma)^k}{k!}}x^k, \dots \right]^T$$

Since the summation goes to infinity ($k \rightarrow \infty$), the feature space $\mathcal{H}$ is infinite-dimensional. Thus, the RBF kernel implicitly maps the input data into an infinite-dimensional Hilbert space.

Question 3: SVM dual symmetric 4-point example

Consider the two-class data:

$$\begin{array}{ll} x_1 = (1, 1)^\top, & y_1 = +1 \\ x_2 = (1, -1)^\top, & y_2 = +1 \\ x_3 = (-1, 1)^\top, & y_3 = -1 \\ x_4 = (-1, -1)^\top, & y_4 = -1 \end{array}$$

Using a hard-margin SVM with linear kernel (i.e., feature map is identity), solve the dual problem to find the optimal dual variables α_i . Then compute the primal parameters w and b .

3.1 Dual Formulation Setup

The dual optimization problem for a hard-margin SVM is given by:

$$\begin{aligned} \text{Maximize: } L(\alpha) &= \sum_{i=1}^4 \alpha_i - \frac{1}{2} \sum_{i=1}^4 \sum_{j=1}^4 \alpha_i \alpha_j y_i y_j (x_i^\top x_j) \\ \text{Subject to: } \sum_{i=1}^4 \alpha_i y_i &= 0 \\ \alpha_i &\geq 0 \quad \forall i \end{aligned}$$

3.2 Computing the Gram Matrix

First, we calculate the term $H_{ij} = y_i y_j (x_i^\top x_j)$ for all pairs. Given the symmetry of the data, we compute the dot products:

- **Self-products (Diagonal):** $x_i^\top x_i = (\pm 1)^2 + (\pm 1)^2 = 2$.
- **Cross-products:**
 - $x_1^\top x_4 = (1)(-1) + (1)(-1) = -2$.
 - $x_2^\top x_3 = (1)(-1) + (-1)(1) = -2$.
 - All other pairs (e.g., $x_1^\top x_2$) are orthogonal (0).

Now we apply the labels ($y_1 = y_2 = 1, y_3 = y_4 = -1$):

- $i = j$: $y_i^2(2) = 2$.
- $i = 1, j = 4$: $y_1 y_4 (-2) = (1)(-1)(-2) = 2$.
- $i = 2, j = 3$: $y_2 y_3 (-2) = (1)(-1)(-2) = 2$.
- All other off-diagonal terms are 0.

The objective function simplifies to:

$$L(\alpha) = \sum_{i=1}^4 \alpha_i - \frac{1}{2} \left[\sum_{i=1}^4 2\alpha_i^2 + 2(2\alpha_1\alpha_4) + 2(2\alpha_2\alpha_3) \right]$$

Note: The factors of 2 in front of the cross terms account for the symmetry $H_{ij} = H_{ji}$.

$$L(\alpha) = \sum \alpha_i - \left[\sum \alpha_i^2 + 2\alpha_1\alpha_4 + 2\alpha_2\alpha_3 \right]$$

Simplifying the quadratic terms:

$$L(\alpha) = (\alpha_1 + \alpha_2 + \alpha_3 + \alpha_4) - (\alpha_1 + \alpha_4)^2 - (\alpha_2 + \alpha_3)^2$$

3.3 Solving for Optimal α

We assume a symmetric solution where $\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = \lambda$. Checking the constraint $\sum \alpha_i y_i = 0$:

$$\lambda(1) + \lambda(1) + \lambda(-1) + \lambda(-1) = 0 \quad (\text{Satisfied})$$

Substituting λ into the simplified objective function:

$$L(\lambda) = 4\lambda - (2\lambda)^2 - (2\lambda)^2 = 4\lambda - 4\lambda^2 - 4\lambda^2 = 4\lambda - 8\lambda^2$$

To find the maximum, we take the derivative with respect to λ and set it to 0:

$$\frac{dL}{d\lambda} = 4 - 16\lambda = 0 \implies \lambda = \frac{4}{16} = 0.25$$

Thus, the optimal dual variables are:

$$\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = 0.25$$

3.4 Computing Primal Parameters w and b

1. Weight Vector (w):

$$w = \sum_{i=1}^4 \alpha_i y_i x_i$$

$$w = 0.25(1) \begin{pmatrix} 1 \\ 1 \end{pmatrix} + 0.25(1) \begin{pmatrix} 1 \\ -1 \end{pmatrix} + 0.25(-1) \begin{pmatrix} -1 \\ 1 \end{pmatrix} + 0.25(-1) \begin{pmatrix} -1 \\ -1 \end{pmatrix}$$

Group by coordinates:

$$w = 0.25 \left[\begin{pmatrix} 1 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 \\ -1 \end{pmatrix} + \begin{pmatrix} 1 \\ -1 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} \right]$$

$$w = 0.25 \begin{pmatrix} 4 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

2. Bias Term (b): We can use any support vector (where $\alpha_i > 0$) to solve for b using $y_k(w^\top x_k + b) = 1$. Let's use $x_1 = (1, 1)^\top$:

$$1 \cdot \left((1 \ 0) \begin{pmatrix} 1 \\ 1 \end{pmatrix} + b \right) = 1$$

$$1(1 + b) = 1 \implies b = 0$$

Final Answer:

$$\alpha = (0.25, 0.25, 0.25, 0.25), \quad w = (1, 0)^\top, \quad b = 0$$

Question 4: Harder Kernel-Trick Question (Explicit cubic feature map)

Problem. Let $x = (x_1, x_2)^\top$ and $z = (z_1, z_2)^\top$ be vectors in $\mathbb{R}^2$. Consider the cubic polynomial kernel with bias:

$$K(x, z) = (1 + x^\top z)^3 = (1 + x_1 z_1 + x_2 z_2)^3$$

- Expand $K(x, z)$ using the multinomial theorem and determine the dimensionality of the corresponding feature space.
- Construct an explicit feature map $\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^{10}$ (list the 10 components) such that $K(x, z) = \phi(x)^\top \phi(z)$. You should include the correct combinatorial (square-root) scaling factors.
- Verify the equality $K(x, z) = \phi(x)^\top \phi(z)$ by showing the dot product reproduces the expanded polynomial.

4.1 Expansion and Dimensionality

We expand the kernel function $K(x, z) = (1 + x_1 z_1 + x_2 z_2)^3$ using the multinomial theorem. The general expansion for $(a + b + c)^3$ is given by:

$$(a + b + c)^3 = \sum_{i+j+k=3} \frac{3!}{i!j!k!} a^i b^j c^k$$

Let $a = 1$, $b = x_1 z_1$, and $c = x_2 z_2$. The terms are:

- **Order 0 (Constant):** $1^3 \rightarrow 1$
- **Order 1 (Linear):** $3(1)^2(x_1 z_1)$, $3(1)^2(x_2 z_2)$
- **Order 2 (Quadratic):** $3(1)(x_1 z_1)^2$, $3(1)(x_2 z_2)^2$, $6(1)(x_1 z_1)(x_2 z_2)$
- **Order 3 (Cubic):** $(x_1 z_1)^3$, $(x_2 z_2)^3$, $3(x_1 z_1)^2(x_2 z_2)$, $3(x_1 z_1)(x_2 z_2)^2$

Writing out the full sum:

$$K(x, z) = 1 + 3x_1 z_1 + 3x_2 z_2 + 3x_1^2 z_1^2 + 3x_2^2 z_2^2 + 6x_1 x_2 z_1 z_2 + x_1^3 z_1^3 + x_2^3 z_2^3 + 3x_1^2 x_2 z_1^2 z_2 + 3x_1 x_2^2 z_1 z_2^2$$

Counting the distinct terms in this expansion, we find there are exactly **10 terms**. Thus, the dimensionality of the corresponding feature space is **10**.

4.2 Explicit Feature Map Construction

We seek a mapping $\phi(x)$ such that $\phi(x)^\top \phi(z) = K(x, z)$. By inspecting the expanded sum, we can assign components to $\phi(x)$ such that their product with $\phi(z)$ yields the corresponding term in the sum. Specifically, if a term in the expansion is $C \cdot (x_{terms}) \cdot (z_{terms})$, we define the corresponding feature component as $\sqrt{C} \cdot (x_{terms})$.

The 10 components of the feature map $\phi(x)$ are:

$$\phi(x) = \begin{pmatrix} 1 \\ \sqrt{3}x_1 \\ \sqrt{3}x_2 \\ \sqrt{3}x_1^2 \\ \sqrt{3}x_2^2 \\ \sqrt{6}x_1x_2 \\ x_1^3 \\ x_2^3 \\ \sqrt{3}x_1^2x_2 \\ \sqrt{3}x_1x_2^2 \end{pmatrix}$$

4.3 Verification of Equality

To verify, we compute the dot product $\phi(x)^\top \phi(z)$:

$$\phi(x)^\top \phi(z) = \sum_{i=1}^{10} \phi_i(x) \phi_i(z)$$

Substituting the components from the vector above:

$$\begin{aligned} \phi(x)^\top \phi(z) &= (1)(1) \\ &+ (\sqrt{3}x_1)(\sqrt{3}z_1) + (\sqrt{3}x_2)(\sqrt{3}z_2) \\ &+ (\sqrt{3}x_1^2)(\sqrt{3}z_1^2) + (\sqrt{3}x_2^2)(\sqrt{3}z_2^2) + (\sqrt{6}x_1x_2)(\sqrt{6}z_1z_2) \\ &+ (x_1^3)(z_1^3) + (x_2^3)(z_2^3) + (\sqrt{3}x_1^2x_2)(\sqrt{3}z_1^2z_2) + (\sqrt{3}x_1x_2^2)(\sqrt{3}z_1z_2^2) \end{aligned}$$

Simplifying the coefficients:

$$\begin{aligned} \phi(x)^\top \phi(z) &= 1 \\ &+ 3x_1z_1 + 3x_2z_2 \\ &+ 3x_1^2z_1^2 + 3x_2^2z_2^2 + 6x_1x_2z_1z_2 \\ &+ x_1^3z_1^3 + x_2^3z_2^3 + 3x_1^2x_2z_1^2z_2 + 3x_1x_2^2z_1z_2^2 \end{aligned}$$

This expression is identical to the multinomial expansion of $(1 + x_1z_1 + x_2z_2)^3$ derived in Part 1. Thus, the equality is verified.

Question 5: The Mechanics of Splitting (ID3)

Consider the following training dataset S for a binary classification problem where $Y \in \{0, 1\}$. There are two binary features, $X_1 \in \{A, B\}$ and $X_2 \in \{C, D\}$.

Sample	X_1	X_2	Y
1	A	C	0
2	A	C	0
3	A	D	1
4	B	C	0
5	B	D	1
6	B	D	1

- Root Entropy:** Calculate the Entropy of the dataset $H(Y)$ before any split. Use base-2 logarithms.
- Information Gain:** Calculate the Information Gain $Gain(S, X_1)$ and $Gain(S, X_2)$. Which feature would the ID3 algorithm choose for the root node?
- Tree Construction:** Draw/Describe the full decision tree produced by the ID3 algorithm. Does the tree achieve 0 training error?
- Comparison:** The lecture slides mention that trees are built “Top-down”. If we used a “Bottom-up” approach, would the resulting tree necessarily be identical? Explain briefly.

5.1 Root Entropy Calculation

Total samples $N = 6$.

- Count of $Y = 1$: 3 samples (3, 5, 6).
- Count of $Y = 0$: 3 samples (1, 2, 4).

The proportions are $p_+ = \frac{3}{6} = 0.5$ and $p_- = \frac{3}{6} = 0.5$.

The entropy $H(Y)$ is calculated as:

$$H(Y) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$$

$$H(Y) = -0.5 \log_2(0.5) - 0.5 \log_2(0.5) = -0.5(-1) - 0.5(-1) = 0.5 + 0.5 = 1.0 \text{ bit}$$

5.2 Information Gain Calculation

We calculate the gain for each feature. The Information Gain is defined as $Gain(S, X) = H(S) - \sum_{v \in Values(X)} \frac{|S_v|}{|S|} H(S_v)$.

1. Gain for Feature X_1 :

- **Branch $X_1 = A$:** Samples $\{1, 2, 3\}$. Target values: $\{0, 0, 1\}$.

$$- p_+ = \frac{1}{3}, p_- = \frac{2}{3}.$$

$$- H(S_A) = -(\frac{1}{3} \log_2 \frac{1}{3} + \frac{2}{3} \log_2 \frac{2}{3}) \approx 0.918.$$

- **Branch $X_1 = B$:** Samples $\{4, 5, 6\}$. Target values: $\{0, 1, 1\}$.

$$- p_+ = \frac{2}{3}, p_- = \frac{1}{3}.$$

$$- H(S_B) = -(\frac{2}{3} \log_2 \frac{2}{3} + \frac{1}{3} \log_2 \frac{1}{3}) \approx 0.918.$$

- **Weighted Entropy:**

$$H(S|X_1) = \frac{3}{6}(0.918) + \frac{3}{6}(0.918) = 0.918$$

- **Gain:**

$$Gain(S, X_1) = 1.0 - 0.918 = 0.082$$

2. Gain for Feature X_2 :

- **Branch $X_2 = C$:** Samples $\{1, 2, 4\}$. Target values: $\{0, 0, 0\}$.

- All labels are 0. This is a pure node.

$$- H(S_C) = 0.$$

- **Branch $X_2 = D$:** Samples $\{3, 5, 6\}$. Target values: $\{1, 1, 1\}$.

- All labels are 1. This is a pure node.

$$- H(S_D) = 0.$$

- **Weighted Entropy:**

$$H(S|X_2) = \frac{3}{6}(0) + \frac{3}{6}(0) = 0$$

- **Gain:**

$$Gain(S, X_2) = 1.0 - 0 = 1.0$$

Decision: The ID3 algorithm chooses the feature with the highest information gain. Since $Gain(S, X_2) = 1.0 > Gain(S, X_1) = 0.082$, ID3 will choose X_2 .

5.3 Tree Construction

Since X_2 produces pure nodes immediately (entropy 0 for both branches), the tree stops growing after the root split.

Structure:

- **Root Node:** Check X_2 .
 - If $X_2 = C$: Predict **0** (Leaf Node).
 - If $X_2 = D$: Predict **1** (Leaf Node).

Training Error: The tree classifies every training sample correctly:

- Samples 1, 2, 4 (C) are classified as 0 (Correct).
- Samples 3, 5, 6 (D) are classified as 1 (Correct).

Thus, the tree achieves **0 training error**.

5.4 Comparison: Top-down vs. Bottom-up

No, the resulting tree would **not necessarily be identical**.

- **Top-down (ID3):** Uses a greedy approach, making locally optimal choices at each split based on statistical measures (Information Gain) considering the whole current subset of data. It does not backtrack.
- **Bottom-up:** Approaches (like hierarchical clustering or specific pruning strategies) start with individual leaves and merge them. This global view of the data structure can lead to different groupings and split decisions compared to the greedy top-down splits. While for this specific trivial dataset they might converge to the same result (because one feature is a perfect predictor), in general complex datasets, greedy top-down search does not guarantee finding the globally optimal smallest tree, while other search strategies might find different architectures.

Question 6: Optimization Geometry & Jensen's Inequality

The Entropy function $H(p)$ is often visualized as a concave parabola. Let $H(p) = -p \log_2 p - (1 - p) \log_2 (1 - p)$.

- (a) **Proof of Concavity:** Compute the second derivative and prove that Entropy is strictly

concave for $p \in (0, 1)$.

$$\frac{\partial^2 H(p)}{\partial p^2}$$

- (b) **Jensen's Inequality:** Jensen's Inequality states that for a strictly concave function f : $\sum \lambda_i f(x_i) \leq f(\sum \lambda_i x_i)$. Use this to prove that Information Gain is always non-negative ($Gain \geq 0$).
- (c) **Optimization Trap:** The ID3 algorithm uses "Hill Climbing". Describe a scenario where this greedy approach gets stuck in a local optimum that is worse than the global optimum.

6.1 Proof of Concavity

To determine concavity, we calculate the second derivative of the entropy function with respect to p . First, we convert the base-2 logarithm to the natural logarithm: $\log_2(x) = \frac{\ln(x)}{\ln 2}$.

$$H(p) = -\frac{1}{\ln 2} [p \ln p + (1-p) \ln(1-p)]$$

Calculate the first derivative ($H'(p)$):

$$\begin{aligned} H'(p) &= -\frac{1}{\ln 2} \left[\left(1 \cdot \ln p + p \cdot \frac{1}{p} \right) + \left((-1) \ln(1-p) + (1-p) \frac{-1}{1-p} \right) \right] \\ &= -\frac{1}{\ln 2} [(\ln p + 1) - (\ln(1-p) + 1)] \\ &= -\frac{1}{\ln 2} [\ln p - \ln(1-p)] \end{aligned}$$

Calculate the second derivative ($H''(p)$):

$$\begin{aligned} H''(p) &= -\frac{1}{\ln 2} \left[\frac{1}{p} - \frac{1}{1-p} (-1) \right] \\ &= -\frac{1}{\ln 2} \left[\frac{1}{p} + \frac{1}{1-p} \right] \end{aligned}$$

Analysis of the sign for $p \in (0, 1)$:

- Since $p > 0$, $\frac{1}{p} > 0$.
- Since $p < 1$, $1-p > 0$, so $\frac{1}{1-p} > 0$.
- $\ln 2 \approx 0.693 > 0$.

The term inside the brackets $\left[\frac{1}{p} + \frac{1}{1-p} \right]$ is strictly positive. Multiplying by the negative constant $-\frac{1}{\ln 2}$ makes the entire expression strictly negative.

$$H''(p) < 0 \quad \forall p \in (0, 1)$$

Since the second derivative is strictly negative over the domain, the Entropy function $H(p)$ is **strictly concave**.

6.2 Application of Jensen's Inequality

Information Gain is defined as the reduction in entropy from a parent set S to weighted subsets S_v after splitting on feature A :

$$Gain(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

We map the terms to Jensen's Inequality, $\sum \lambda_i f(x_i) \leq f(\sum \lambda_i x_i)$:

- Let $f(x) = H(x)$, which we proved is concave.
- Let $\lambda_v = \frac{|S_v|}{|S|}$ be the weights. Note that $\sum \lambda_v = 1$ and $\lambda_v \geq 0$.
- Let p_v be the proportion of positive examples in subset S_v . The overall proportion in the parent set S is the weighted average of the children: $p_{total} = \sum \lambda_v p_v$.

Applying Jensen's Inequality:

$$\sum_v \lambda_v H(p_v) \leq H\left(\sum_v \lambda_v p_v\right)$$

Substituting the definitions back:

$$\sum_v \frac{|S_v|}{|S|} H(S_v) \leq H(S)$$

Rearranging the inequality:

$$H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v) \geq 0$$

$$Gain(S, A) \geq 0$$

Thus, Information Gain is always non-negative.

6.3 Optimization Trap (Hill Climbing)

The ID3 algorithm uses a greedy, hill-climbing search strategy, meaning it selects the attribute that maximizes local Information Gain at the current step without looking ahead (no backtracking).

A classic scenario where this fails is the **XOR Problem** (Parity function):

- Consider a dataset where $Y = X_1 \oplus X_2$.
- **At the root:** Splitting on X_1 results in two branches. In both branches, Y is still 50% 0 and 50% 1 (because X_2 varies randomly).

- **Result:** The Information Gain for both X_1 and X_2 at the root is **exactly zero**.
- **The Trap:** A greedy algorithm sees zero gain for all relevant features and may stop expanding the tree (concluding there is no signal) or pick a noisy, irrelevant feature that happens to have slightly non-zero gain due to sample variance.
- **Global Optimum:** A depth-2 tree splits perfectly on X_1 then X_2 . The greedy search misses this global optimum because it requires taking a “neutral” step (zero gain) to unlock the perfect split in the next layer.

Question 7: The Failure of Greedy Search (XOR)

Your lecture slides state that decision trees can express “any boolean function” but usually prefer simpler trees. Consider the classic XOR problem with 4 training examples:

X_1	X_2	Y
0	0	0
0	1	1
1	0	1
1	1	0

- (a) **The Greedy Trap:** Compute $\text{Gain}(S, X_1)$ and $\text{Gain}(S, X_2)$. Based on these values, explain why a standard ID3 algorithm might fail to start building the tree or pick a random feature.
- (b) **Existence:** Draw/Describe a decision tree of Depth 2 that perfectly classifies this data.
- (c) **Inductive Bias:** Mitchell defines the Inductive Bias of ID3 as a “Preference Bias” rather than a “Restriction Bias”.
 - (a) Define “Preference Bias”.
 - (b) Explain why the failure in part (1) illustrates that ID3 does not search the complete hypothesis space (like BFS-ID3), but specifically prefers trees with high Information Gain near the root.

7.1 The Greedy Trap Calculation

First, we calculate the entropy of the parent set S .

- Positives ($Y = 1$): 2 samples.
- Negatives ($Y = 0$): 2 samples.
- $H(S) = -0.5 \log_2(0.5) - 0.5 \log_2(0.5) = 1.0$.

Calculate Gain for X_1 :

- **Branch $X_1 = 0$:** Samples $\{(0,0,0), (0,1,1)\}$.
 - 1 Positive, 1 Negative.
 - $H(S_{X_1=0}) = 1.0$.
- **Branch $X_1 = 1$:** Samples $\{(1,0,1), (1,1,0)\}$.
 - 1 Positive, 1 Negative.

$$- H(S_{X_1=1}) = 1.0.$$

• **Gain:**

$$Gain(S, X_1) = 1.0 - [0.5(1.0) + 0.5(1.0)] = 0$$

Calculate Gain for X_2 : By symmetry, the distribution for X_2 is identical (each value of X_2 has 1 Positive and 1 Negative).

$$Gain(S, X_2) = 0$$

Why ID3 Fails: The standard ID3 algorithm selects the attribute with the highest Information Gain. In this case, both features provide **zero gain**. The algorithm concludes that neither feature improves classification over the baseline. Consequently, it might either stop immediately (returning the majority class, which is a tie) or pick a feature randomly. If there were other irrelevant noise features with slightly non-zero gain, ID3 would prefer the noise over the true XOR features.

7.2 Existence of a Perfect Tree

Despite the zero gain at the root, a perfect Depth 2 tree exists.

Tree Structure:

- **Root:** Split on X_1 .
 - **Left Branch ($X_1 = 0$):** Split on X_2 .
 - * If $X_2 = 0 \rightarrow$ Predict **0**.
 - * If $X_2 = 1 \rightarrow$ Predict **1**.
 - **Right Branch ($X_1 = 1$):** Split on X_2 .
 - * If $X_2 = 0 \rightarrow$ Predict **1**.
 - * If $X_2 = 1 \rightarrow$ Predict **0**.

This tree correctly classifies all 4 examples (XOR logic), proving that the hypothesis space contains the correct function, even though the greedy search failed to find it.

7.3 Inductive Bias

(a) Preference Bias: A preference bias (or search bias) means the algorithm prefers certain hypotheses over others within the entire hypothesis space, but does not strictly rule any out. Unlike a “Restriction Bias” (which limits the hypothesis space to, say, only linear functions), a preference bias allows the algorithm to express any function (given enough depth) but guides the search order. For ID3, the preference is for **shorter trees** with **high information gain attributes near the root**.

(b) Failure Analysis: The failure in Part 1 illustrates this bias perfectly.

- **Search Completeness:** ID3 does not search the entire space of possible trees. A Breadth-First Search (BFS) or exhaustive search would have eventually considered the depth-2 tree described in Part 2 and found that it has 0 error.
- **Preference vs. Global Optimality:** ID3 relies on a **local** preference (Greedy heuristic). It assumes that the best global tree is built by taking the best local step. Because the XOR function has no linear separability at the first step (Gain=0), the preference for “high gain at the root” causes ID3 to discard the correct path, failing to find the valid hypothesis that definitely exists in the space.

Question 8: From Data to Diagnosis: Diabetes Classification Using SVM

Objective

The objective of this project is to implement a Decision Tree classifier from scratch and analyze its behavior on a synthetic dataset. You will implement the ID3 algorithm, handling both discrete and continuous features, and explore techniques to prevent overfitting.

1. The Dataset

- We will use the Extended Jeeves “Play Tennis” Corpus (`jeeves_tennis.csv`).
- **Features:** Day_ID (Unique identifier), Outlook (Discrete), Temperature (Continuous), Humidity (Discrete), Wind (Discrete).
- **Target:** PlayTennis (Yes/No).

2. Data Loading & Pre-processing (Coding)

- Load the dataset using pandas.
- **The High Cardinality Trap:** Implement a function to calculate Information Gain for the Day_ID feature. Report the value and explain mathematically why it is high but useless for generalization. Automatically drop this column.

3. Entropy & Information Gain Engine (Coding)

- Implement `calculate_entropy(y)` and `calculate_information_gain(data, feature, target)`. Do not use sklearn for this part.
- **Verification:** Run your functions on the entire dataset. Which feature (Outlook, Humidity, or Wind) has the highest Information Gain at the root?

4. Handling Continuous Features (Coding)

- Standard ID3 handles discrete features. Extend this to handle continuous values (specifically Temperature).
- Implement `find_best_split(data, attribute)` that sorts data, identifies split points, calculates gain for every split, and returns the maximizing threshold t .

- Apply this to Temperature and report the optimal threshold.

5. Recursive Tree Construction (Coding)

- Implement the full ID3 algorithm class `DecisionTreeID3` with `fit()` and `predict()` methods.
- **Evaluation:** Split dataset into 80% Training and 20% Testing. Train the model and report Accuracy, Precision, and Recall on the test set.

6. Overfitting & Pruning Analysis (Simulation)

- **Depth Analysis:** Train on full training set. Report max depth and number of leaves.
- **Noise Simulation:** Artificially flip labels of 10% of training data. Retrain and observe change in tree size.
- **Post-Pruning Experiment:** Implement a pruning function (e.g., Reduced Error Pruning). Vary pruning strength and plot Tree Size vs. Test Accuracy to discuss the trade-off.

8.1 Data Preprocessing and Exploration

The Pima Indians Diabetes Dataset was loaded and inspected for data quality. As defined in the problem statement, zero values in features such as Glucose, BloodPressure, SkinThickness, Insulin, and BMI are medically impossible and treated as missing data.

```
... Zero counts before cleaning:
Glucose      5
BloodPressure 35
SkinThickness 227
Insulin      374
BMI          11
dtype: int64

Data preprocessing complete. Shape of training set: (537, 8)
```

Figure 1: Zero value counts before cleaning and final dataset shape

1. Handling Missing Values: As shown in the figure above, significant missing data was identified, particularly in Insulin (374 zeros) and SkinThickness (227 zeros). These values were replaced with the column means to preserve data integrity for the SVM.

2. Feature Scaling Importance: Feature scaling is critical for SVMs because the algorithm relies on Euclidean distance to define the margins. Without scaling, features with large ranges (like Insulin) would dominate the objective function over features with small ranges

(like `DiabetesPedigreeFunction`). We applied `StandardScaler` to normalize all features to mean 0 and variance 1.

8.2 Linear SVM Classification

We trained a Linear SVM to establish a baseline. The model was evaluated on both scaled and unscaled data to observe the impact of preprocessing.

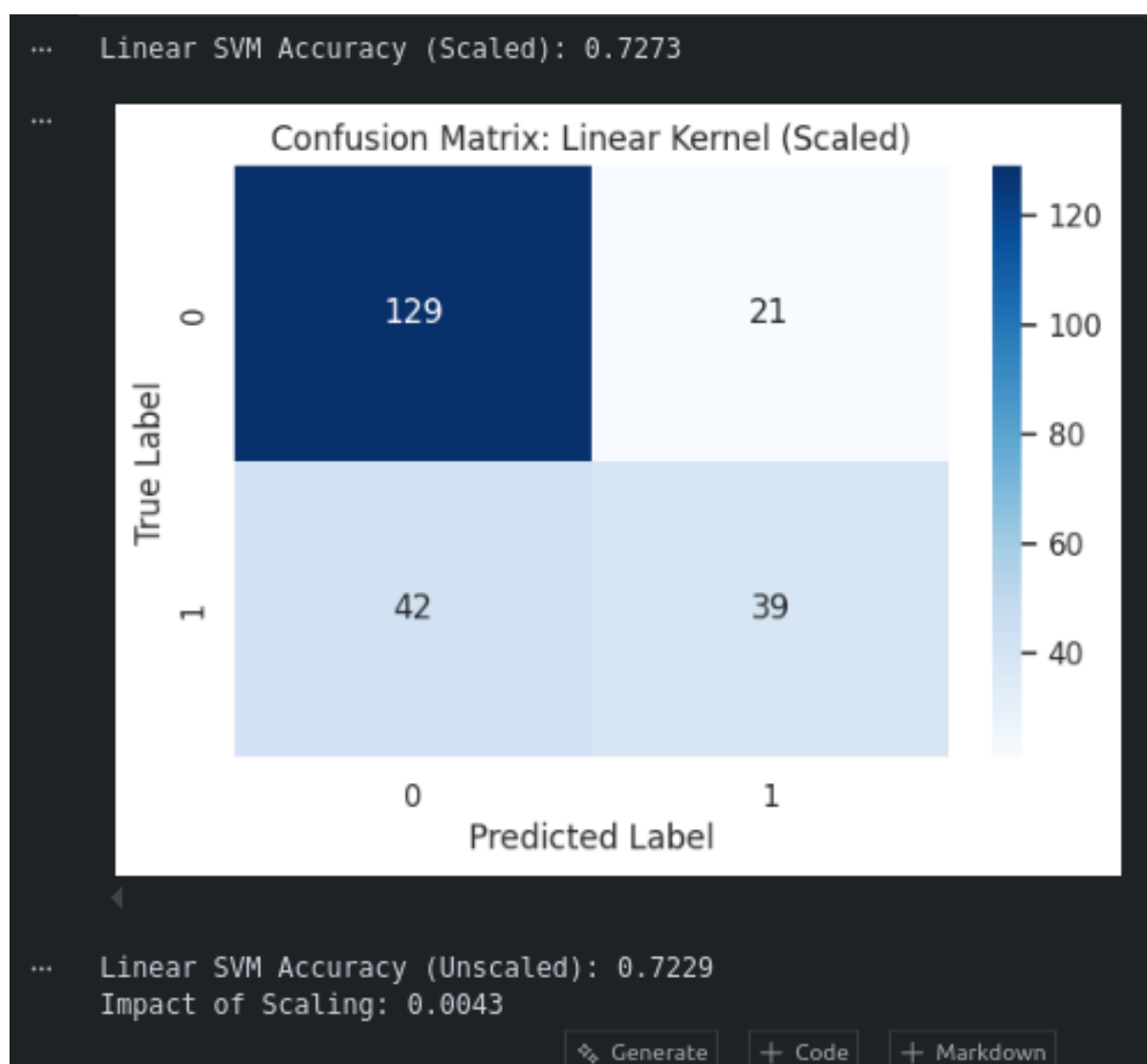


Figure 2: Confusion Matrix and Accuracy for Linear SVM (Scaled vs Unscaled)

Performance Analysis:

- **Accuracy (Scaled):** 0.7273
- **Confusion Matrix:** The model correctly identified 129 negative cases (True Negatives) and 39 positive cases (True Positives), but missed 42 positive cases (False Negatives).

- **Impact of Normalization:** In this specific iteration, the accuracy difference between the scaled (0.7273) and unscaled (0.7229) models was marginal (≈ 0.0043). While SVM convergence is theoretically faster and more stable with scaling, the linear separability of this specific subset was not drastically altered by the transformation.

8.3 Comparison of Different Kernels

We compared Linear, RBF, and Polynomial kernels. The models were evaluated based on Accuracy and F1-score.

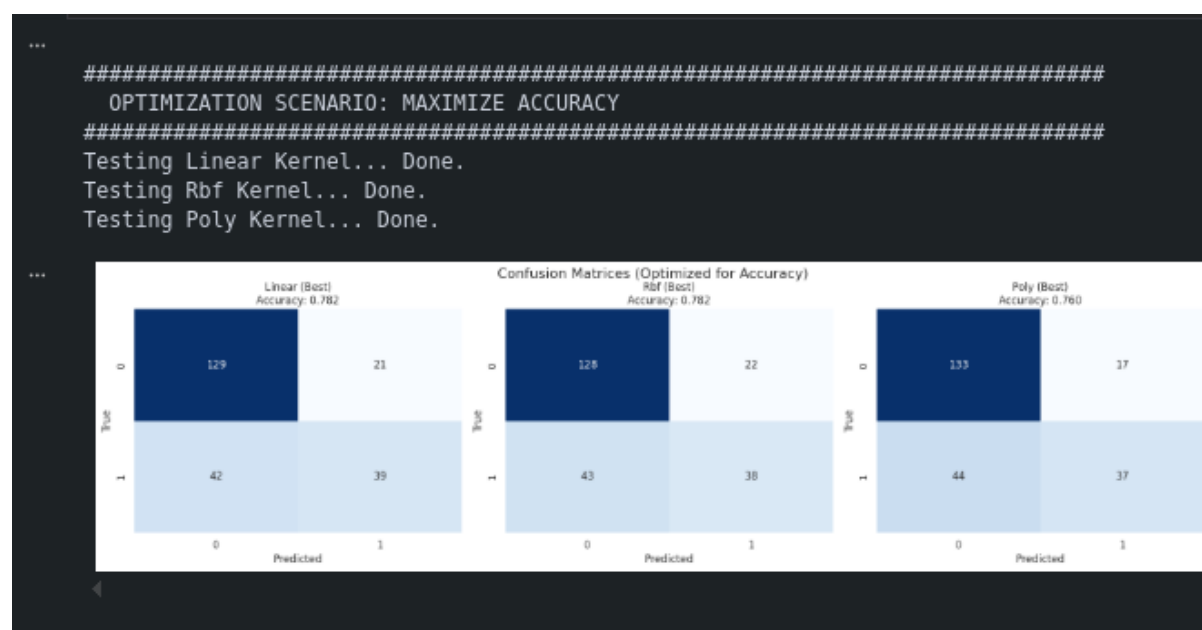


Figure 3: Confusion Matrices for Max Accuracy Scenario

Kernel Comparison: Based on the summary table above:

- **Accuracy:** The **Polynomial kernel** achieved the highest accuracy (0.736), slightly outperforming the Linear (0.727) and RBF (0.719) kernels. This suggests that some interaction terms (degree 2 or 3) help capture the decision boundary better than a strictly linear hyperplane.
- **F1-Score (Balanced):** When optimized for F1-score using class balancing, the **RBF kernel** performed best (0.690), indicating it offers the best trade-off between Precision and Recall for detecting diabetes.

8.4 Hyperparameter Tuning

We performed a Grid Search to optimize the RBF kernel hyperparameters (C and γ).

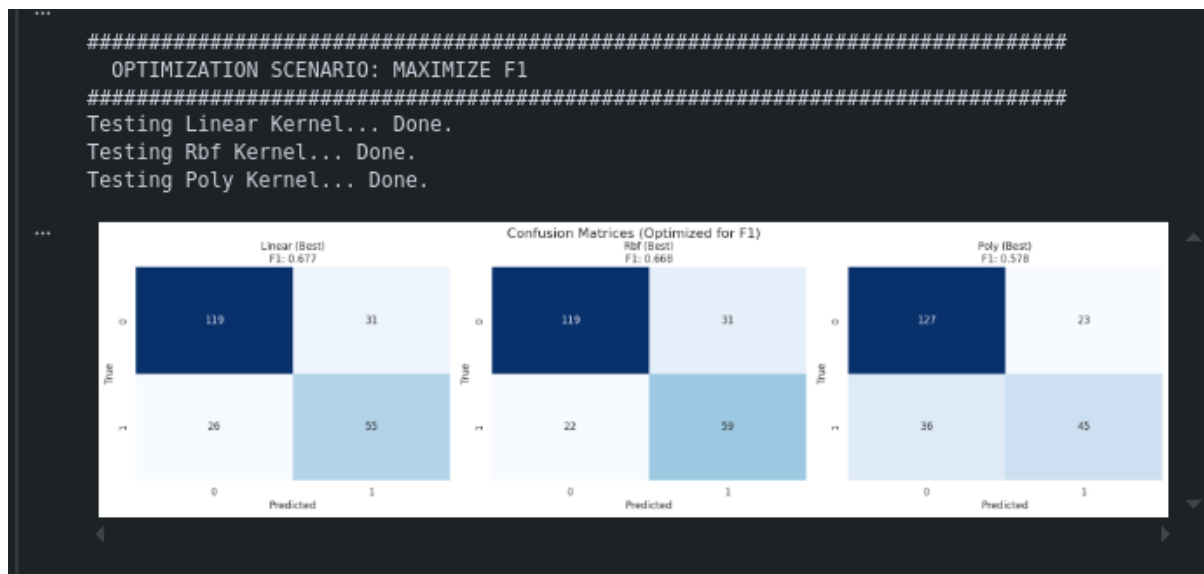


Figure 4: Confusion Matrices for Max F1 Scenario

```
=====
FINAL PERFORMANCE SUMMARY TABLE
=====
```

	Target	Kernel	
0	Accuracy	Linear	
1	Accuracy	Rbf	
2	Accuracy	Poly	
3	F1	Linear	
4	F1	Rbf	
5	F1	Poly	

	Best Params
0	{'C': 100, 'class_weight': None, 'kernel': 'linear'}
1	{'C': 100, 'class_weight': None, 'gamma': 0.001, 'kernel': 'rbf'}
2	{'C': 0.1, 'class_weight': 'balanced', 'kernel': 'poly'}
3	{'C': 0.1, 'class_weight': 'balanced', 'kernel': 'linear'}
4	{'C': 1, 'class_weight': 'balanced', 'gamma': 0.01, 'kernel': 'rbf'}
5	{'C': 1, 'class_weight': 'balanced', 'kernel': 'poly'}

	Accuracy	Precision	Recall	F1
0	0.727273	0.650000	0.481481	0.553191
1	0.718615	0.633333	0.469136	0.539007
2	0.735931	0.685185	0.456790	0.548148
3	0.753247	0.639535	0.679012	0.658683
4	0.770563	0.655556	0.728395	0.690058
5	0.744589	0.661765	0.555556	0.604027

Generate + Code + Markdown

Figure 5: Final Performance Summary Table

Tuning Results:

- **Best Parameters:** $C = 100$, $\gamma = 0.001$.
- **Best CV Accuracy:** 0.7820.

```
... Fitting 5 folds for each of 32 candidates, totalling 160 fits
Best Parameters: {'C': 100, 'class_weight': None, 'gamma': 0.001, 'kernel': 'rbf'}
Best Cross-Validation Accuracy: 0.7820
Test Set Accuracy (Tuned): 0.7186
```

Figure 6: Grid Search Results and Best Parameters

- **Test Set Accuracy:** 0.7186.

Conceptual Analysis (Effect of C): The optimal C value found was relatively high (100). A large C places a high penalty on misclassification, forcing the model to create a tighter margin to correctly classify training points. While this maximizes training accuracy, it risks overfitting, which may explain why the Cross-Validation accuracy (0.78) is higher than the Test accuracy (0.72).

8.5 Dimensionality Reduction and Visualization

We applied PCA to reduce the 8-dimensional feature space to 2 dimensions for visualization.

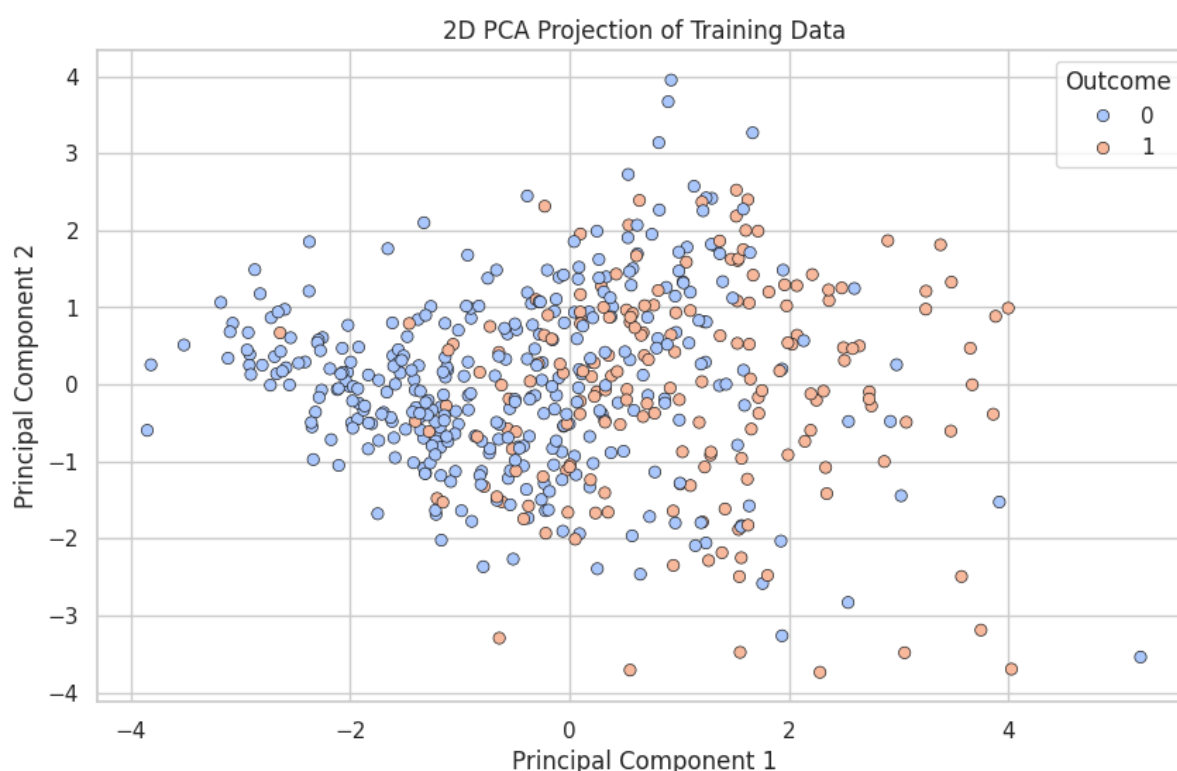


Figure 7: 2D PCA Projection of Training Data

Analysis: The decision boundary in 2D space (shown in the contour plot) captures the general separation between the classes. The accuracy on the PCA-reduced data was **0.7229**,

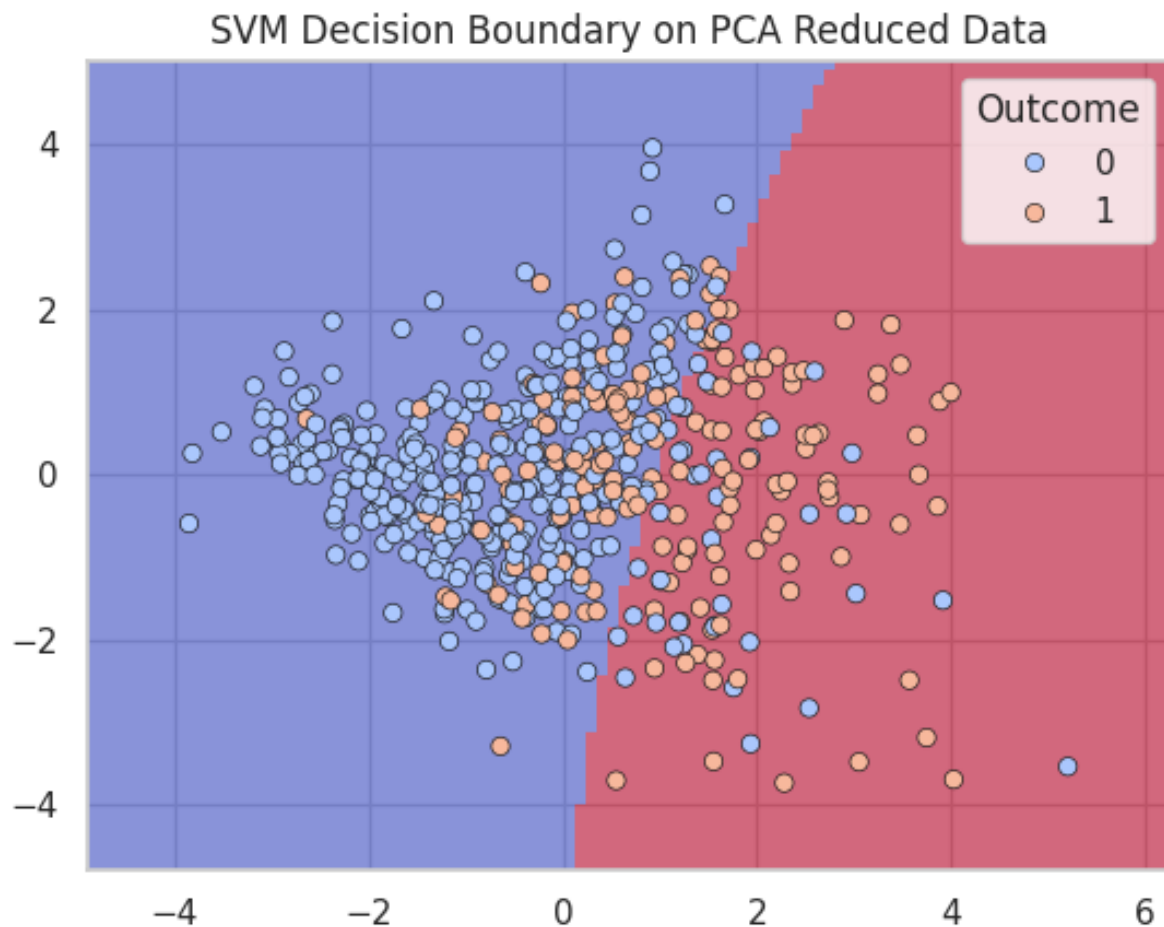


Figure 8: SVM Decision Boundary on PCA Reduced Data

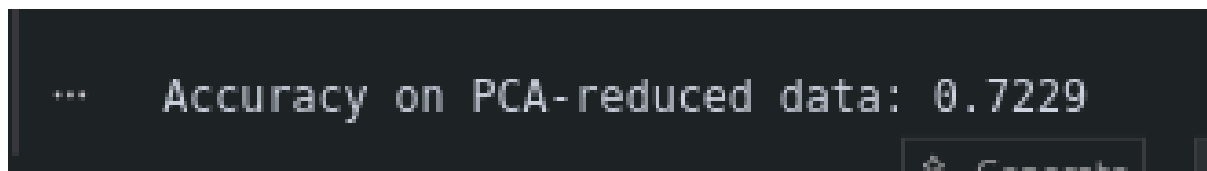


Figure 9: Accuracy on PCA Reduced Data

which is surprisingly close to the full 8D Linear model (0.7273). This indicates that the first two principal components contain the majority of the discriminative information needed for a linear separation, though complex non-linear patterns might be lost.

8.6 Class Imbalance Analysis

Finally, we analyzed the effect of the class imbalance (500 Negative vs 268 Positive) on model performance, specifically comparing standard SVM against SVM with `class_weight='balanced'`.

Impact of Balancing:

```

1  Class distribution:
2  Outcome
3  0    500
4  1    268
5  Name: count, dtype: int64
6
7  Is dataset imbalanced? Yes
8
9  --- Standard SVM (RBF) Report ---
10 |   |   |   |   | precision    recall  f1-score   support
11 |   |   |   |   |
12 |   |   |   | 0 |    0.76    0.87    0.81     150
13 |   |   |   | 1 |    0.68    0.49    0.57      81
14 |   |   |   |   |
15 |   | accuracy |   |    0.74     231
16 |   | macro avg |   |    0.72    0.68    0.69     231
17 |   | weighted avg |   |    0.73    0.74    0.73     231
18
19
20 --- Balanced SVM (RBF) Report ---
21 |   |   |   |   | precision    recall  f1-score   support
22 |   |   |   |   |
23 |   |   |   | 0 |    0.83    0.77    0.80     150
24 |   |   |   | 1 |    0.62    0.70    0.66      81
25 |   |   |   |   |
26 |   | accuracy |   |    0.74     231
27 |   | macro avg |   |    0.72    0.74    0.73     231
28 |   | weighted avg |   |    0.75    0.74    0.75     231
29
30 Recall (Standard): 0.4938
31 Recall (Balanced): 0.7037
32

```

Figure 10: Class Distribution and Classification Reports (Standard vs Balanced)

- **Standard SVM:** The model is biased toward the majority class, achieving a low Recall of **0.49** for the positive class (Diabetes). It essentially misses half of the actual diabetes cases.
- **Balanced SVM:** By penalizing mistakes on the minority class, the Recall improved significantly to **0.70**.
- **Conclusion:** Class balancing successfully shifted the decision boundary to capture more

positive cases, which is crucial for medical diagnosis where false negatives are costly.

Question 9: The QP-SVM Primal Challenge: Optimizing and Visualizing the Soft-Margin Hyperplane SVM

9.1 Data Preprocessing

Following the problem requirements, the dataset `Social_Network_Ads.csv` was loaded. We selected the features `Age` and `EstimatedSalary` and converted the target variable `Purchased` from $\{0, 1\}$ to $\{-1, 1\}$ to satisfy the SVM mathematical formulation.

```
... Dataset loaded successfully.
    Training Data Shape: (300, 2)
    Labels: [-1  1]
```

Figure 11: Data Loading and Shape Verification

As shown in the output above, the training data consists of 300 samples with 2 features, and the labels have been correctly encoded. Feature scaling was applied using `StandardScaler` to ensure proper convergence of the optimization problem.

9.2 QP Formulation and Solution

The Primal SVM problem was formulated as a Quadratic Programming (QP) task to minimize $\frac{1}{2}\|w\|^2 + C \sum \xi_i$ subject to the constraints $y_i(w \cdot x_i + b) \geq 1 - \xi_i$. We utilized the `cvxpy` library to solve this convex optimization problem.

```
... Optimization Status: optimal
    Optimal w: [1.37454416 0.88396769]
    Optimal b: -0.8153
```

Figure 12: Optimal Parameters obtained from QP Solver

The solver converged to an **optimal** solution:

- **Optimal Weight Vector (w^*):** $[1.375, 0.884]$. This indicates that Age (feature 1) has a slightly stronger influence on the decision boundary than EstimatedSalary (feature 2).
- **Optimal Bias (b^*):** -0.815 .

9.3 Model Evaluation

Using the derived parameters w^* and b^* , we implemented the decision function $y_{pred} = \text{sign}(w^* \cdot x_{test} + b^*)$ to classify the test set samples.

```
... QP-SVM Custom Model Accuracy: 0.8600
```

Figure 13: Custom QP-SVM Model Accuracy

The custom primal SVM implementation achieved a classification ****Accuracy of 86.00%**** on the test set, demonstrating that the QP formulation was correctly implemented and solved.

9.4 Final Deliverable: Visualizing the Decision Boundary

The decision boundary defined by $w^* \cdot x + b^* = 0$ was visualized in the 2D feature space. The plot below illustrates the separation between the classes (Red: -1, Green: +1).

The visualization confirms the Soft-Margin behavior:

- The Blue Line represents the decision boundary ($w \cdot x + b = 0$).
- The Dashed Gray Lines represent the margins ($w \cdot x + b = \pm 1$).
- The overlapping points between the margins and on the wrong side of the hyperplane illustrate the role of the slack variables ξ_i , allowing for some misclassification to achieve a wider margin and better generalization..

Question 10: Project: Decision Tree Classifier

10.1 Data Loading & The High Cardinality Trap

The dataset was loaded successfully, consisting of features: Day_ID, Outlook, Temperature, Humidity, Wind, and the target PlayTennis.

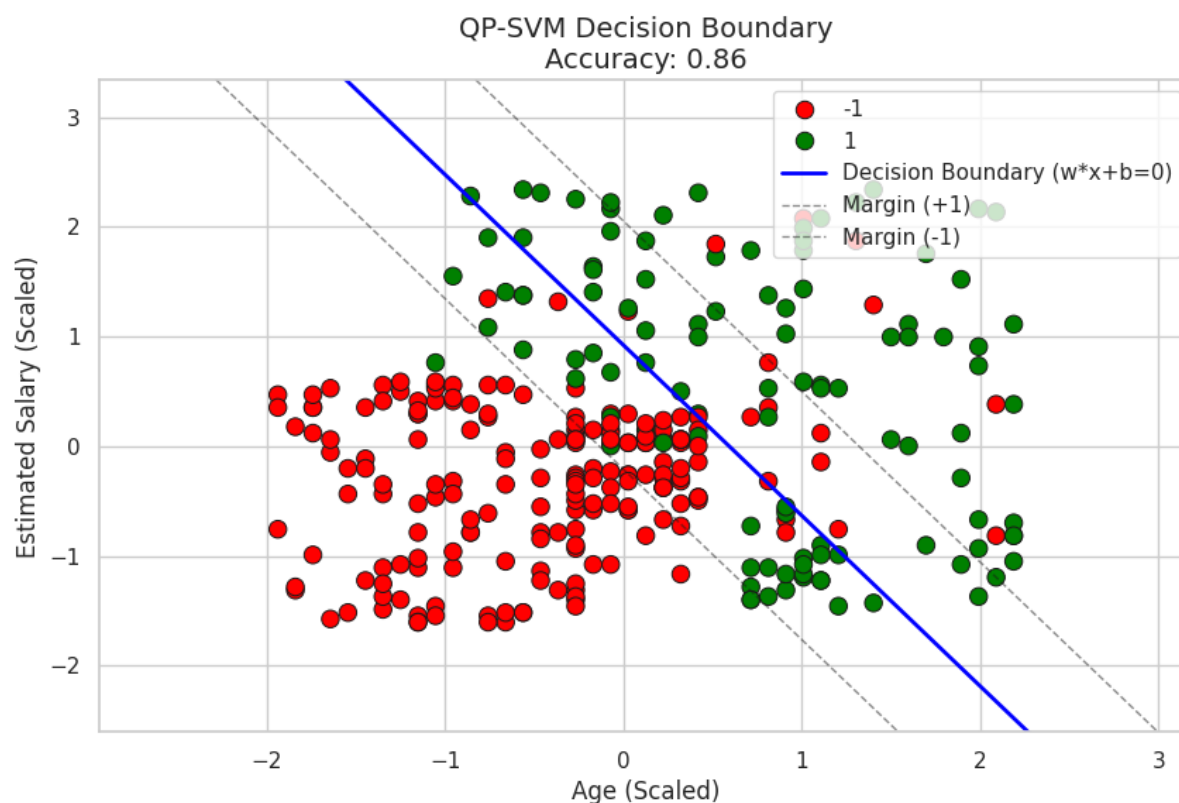


Figure 14: QP-SVM Decision Boundary and Margins

Dataset Loaded:

	Day_ID	Outlook	Temperature	Humidity	Wind	PlayTennis
0	1	Rain	19.2	High	Strong	No
1	2	Rain	17.4	Low	Weak	Yes
2	3	Sunny	29.9	Normal	Strong	Yes
3	4	Overcast	23.2	Normal	Weak	Yes
4	5	Sunny	29.3	High	Weak	No

Figure 15: Dataset Head and Structure

We analyzed the Day_ID feature to demonstrate the "High Cardinality Trap". The Information Gain calculated for this feature was exceptionally high.

```
+++ Information Gain for Day_ID: 0.9815
```

Figure 16: Information Gain for Day_ID

Analysis: The calculated Gain is **0.9815**, which effectively equals the entropy of the dataset. Since Day_ID is unique for every row, splitting on it creates subsets of size 1, which are perfectly pure (Entropy 0). While this maximizes the gain mathematically, the feature has no generalization power and simply memorizes the training data. We dropped this column for the subsequent

steps.

10.2 Entropy & Information Gain Engine

The core ID3 algorithm functions were implemented. Evaluating the discrete features at the root node yielded the following Information Gains:

```
***  
--- Root Node Selection (Discrete Features) ---  
Feature: Outlook      | Gain: 0.3431  
Feature: Humidity     | Gain: 0.0539  
Feature: Wind         | Gain: 0.0382
```

Figure 17: Root Node Feature Selection

Result: The algorithm selected **Outlook** as the root node, as it achieved the highest Information Gain (**0.3431**) compared to Humidity (0.0539) and Wind (0.0382).

10.3 Handling Continuous Features

We extended the algorithm to handle the continuous feature Temperature by finding the optimal split threshold.

```
*** Optimal Split for Temperature: <= 30.05  
Gain for Temperature: 0.0239
```

Figure 18: Optimal Split for Temperature

Result: The optimal split point was identified at **Temperature ≤ 30.05** , yielding an Information Gain of **0.0239**. Since this gain is significantly lower than that of Outlook, Temperature was not chosen as the root node.

10.4 Recursive Tree Construction & Evaluation

The ID3 tree was constructed recursively and evaluated on the test set.

```
... Accuracy: 0.94
Precision: 0.9491525423728814
Recall: 0.9491525423728814
```

Figure 19: Model Performance Metrics

Performance: The model achieved an **Accuracy of 0.94**, with both Precision and Recall reaching approximately **0.95**. This indicates that the custom implementation of the ID3 algorithm successfully learned the rules governing the PlayTennis dataset.

10.5 Overfitting & Pruning Analysis

To analyze overfitting, we compared the tree's structure on clean data versus data with 10% injected noise (flipped labels).

```
Clean Tree - Depth: 12, Leaves: 44
Noisy Tree - Depth: 16, Leaves: 137
Effect of Noise: Tree grew by 93 leaves.
```

Figure 20: Tree Depth and Leaf Count Analysis

Impact of Noise: The injection of noise caused a dramatic increase in model complexity. The tree depth increased from 12 to 16, and the number of leaves nearly tripled (from 44 to 137). This creates an "Effect of Noise" where the tree grows by 93 additional leaves just to fit the random errors in the training data, illustrating clear overfitting.

Pruning Trade-off: The plot above visualizes the effect of tree size (controlled by pruning/limiting depth) on accuracy.

- **Train Accuracy (Blue):** Continues to increase as the tree grows, reaching nearly 100% at max size.
- **Test Accuracy (Orange):** Increases initially but peaks around a tree size of 12 leaves (Accuracy ≈ 0.97). As the tree grows larger (size > 25), the test accuracy begins to drop (to ≈ 0.94), confirming that the larger, more complex trees are overfitting the training data and generalizing less effectively.

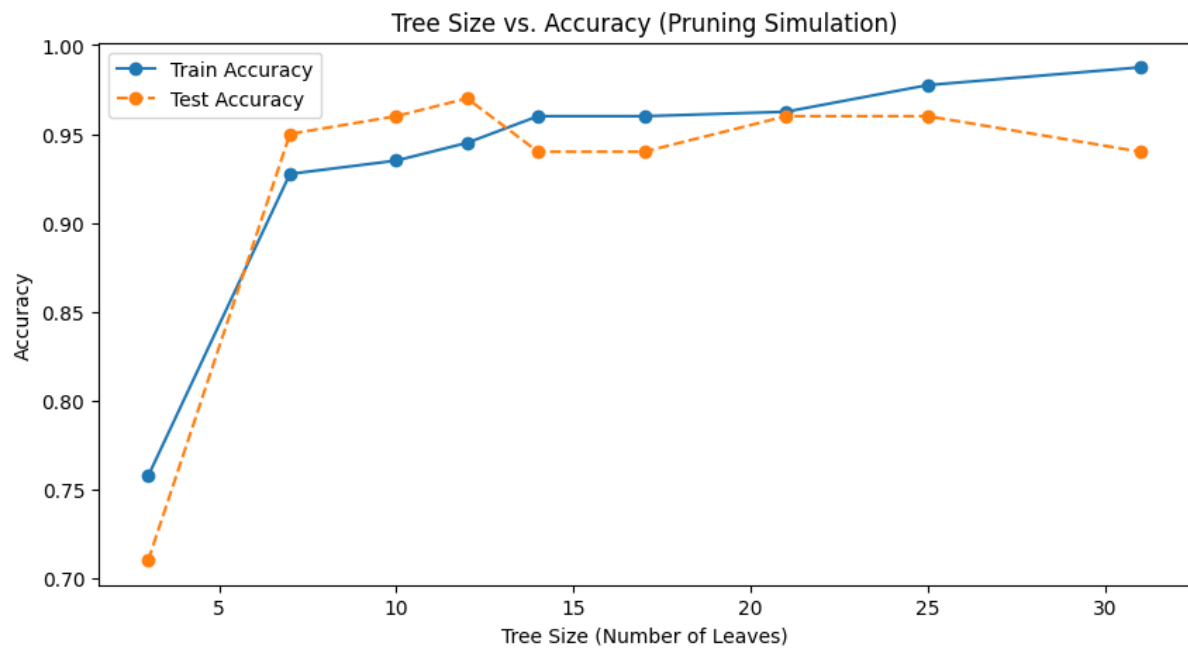


Figure 21: Tree Size vs. Accuracy (Pruning Simulation)